SFDB: A Context-Indexed Semantic Fact Database with Stalk Indexing and Verified Cross-Engine Equivalence

BlackSamuron0305

Abstract

Semantic knowledge is typically represented as collections of subject–predicate–object triples. This representation is elegant for binary relationships, but it forces every higher-arity fact and every context-scoped assertion to be decomposed through reification, which multiplies the number of stored records and turns simple lookups into multi-way joins. We investigate whether a database whose primary organising principle is the context poset, rather than the triple, can offer a more direct representation for facts that are inherently contextual or high-arity.

We describe the Semantic Fact Database (SFDB), a prototype system that stores each fact as a section over a finite poset of contexts and evaluates queries through stalk-indexed retrieval rather than triple reconstruction. The mathematical model is drawn from the language of presheaves on finite posets equipped with the Alexandrov topology. In this setting the sheaf condition holds vacuously, so we do not claim the sheaf axioms as our technical contribution. What the model gives us is a disciplined account of restriction, locality, and gluing that maps directly onto the operational components of the database: the context index, the stalk index, and the global-section constructor. In parallel we implement a conventional reified triple-store baseline over the same core data model and prove semantic equivalence between the two engines through a canonical intermediate representation, which every benchmark run verifies query by query.

The evaluation compares both engines on four query classes (LOOKUP, GLOBAL, bounded TEMPORAL, and unbounded TEMPORAL) plus insert throughput, at scales of 100, 1 000, and 10 000 facts, with every query at every scale checked for cross-engine result equivalence as part of the same run. SFDB is faster than the KG baseline on every measured dimension: inserts by 1.05–1.36×; entity-anchored LOOKUP queries by 23–32×, because the stalk index returns complete facts in a single hash lookup instead of reconstructing them from reified triples through joins; unrestricted full-scan (GLOBAL) queries by 44–95×, once the query path walks the same flat stalk index rather than enumerating topology open sets; bounded temporal-interval queries by 1.2–1.4× via a year-bucketed index; and unbounded temporal-interval queries (open-ended ranges the bucketed index alone cannot narrow) by 1.1–1.2× via a second, flat, binary-searchable temporal index built alongside it — a case we originally reported as an open architectural gap and have since closed (Section 12.2). This is not a uniformly favourable result: SFDB's in-memory indexing structure uses 1.3–2.5× more resident memory per fact than the KG baseline's more compact reified-triple storage at the two larger scales measured, and we report this tradeoff plainly. We also report, as a direct consequence of tightening the cross-engine verification harness during this evaluation, that an earlier version of the sheaf engine silently answered entity-anchored LOOKUP queries with neighbourhood-style (any-mention) semantics rather than strict subject match; the stricter verification caught the discrepancy before publication, which we take as evidence for the value of the equivalence-checking methodology this paper describes, not merely for the numbers it produced.

The contribution of this paper is therefore not a claim of universal performance superiority nor a novel piece of category theory. It is (i) a workable design for a context-indexed semantic fact database backed by an implementation and full test suite, (ii) a mechanised proof of semantic equivalence between the sheaf-inspired engine and a conventional triple store, enforced by an automated verification harness that we show catches real cross-engine

semantic bugs, and (iii) a careful benchmark that maps the design's latency advantages against its memory cost, including closing a genuine architectural gap (unbounded temporal ranges) once it was identified. All results are reproducible from the released artifact with a single command.

1 Introduction

Knowledge graphs have become the default representation for semantic information on the web and in enterprise systems. Their foundations are simple: every fact is a triple (s, p, o) that asserts a binary relationship between a subject, a predicate, and an object [7, 8]. The model is uniform, transparent, and has an enormous accumulated tooling base. It is also a poor fit for large classes of real data.

Two mismatches recur across domains. First, many facts are not binary. A drug interaction record carries a drug, a target, a dosage, a route, a study population, and a confidence interval, all in one statement. A financial transaction pairs a buyer, a seller, an instrument, a price, a currency, a venue, and a timestamp. Encoding such facts as triples requires *reification*: an auxiliary event node is minted, each attribute becomes its own triple pointing at that node, and reconstructing the original fact costs one join per attribute. Second, the same fact often holds in some contexts and not in others. A biological interaction that is valid *in vitro* may fail *in vivo*; a corporate policy that applies in the European subsidiary may not apply in North America. Context is not first-class in the triple model. It is added either through separate named graphs [3], through further reification, or through ad-hoc conventions in the predicate URI.

Both problems have well-known workarounds, and mature triple stores handle them competently at scale. The question we pose in this paper is whether there is anything to gain from a database whose primary organising principle is not the triple but the *context*. Concretely, we ask whether one can build a storage engine that (i) stores each fact as an atomic record, without reification, and (ii) indexes those records by the context poset in a way that answers entity-centric queries via a direct index lookup while remaining verifiably equivalent to a conventional RDF-style store on the queries that both engines can answer.

The mathematical vocabulary that fits this question naturally is that of presheaves on a poset of contexts. A presheaf assigns a set of local sections—in our setting, facts—to each context, and equips the assignment with restriction maps that specialise a fact from a broader context to a narrower one. A sheaf is a presheaf that satisfies two additional axioms of locality and gluing. We stress at the outset that on a finite poset endowed with the Alexandrov topology, these axioms hold vacuously. The system we describe is therefore a *presheaf-based* database in the strict technical sense; the sheaf terminology is retained because the operational vocabulary of restriction, stalks, and global sections is what our implementation actually exposes, not because we claim a non-trivial sheaf-theoretic theorem.

Contributions

The contributions of this paper are, in order:

1. A concise formal model of a context-indexed semantic fact store, expressed in the language of presheaves on a finite poset (Section 6). The model is deliberately minimal: it names only the structure that the implementation uses.

2. A prototype implementation, the *Semantic Fact Database* (SFDB), that realises the model in roughly 15 000 lines of Python (Section 10). Two storage engines are provided over a shared canonical schema: a reference triple store with SPO/POS/OSP indexes and reification for n-ary facts, and a sheaf-inspired engine with a stalk index, a restriction cache, and an on-demand global-section constructor.

3. A canonical intermediate representation together with a bidirectional mapping to and from each engine's native format, used to establish semantic equivalence at every insert and every query (Section 7). Cross-engine equivalence is enforced by the test suite (358 passing tests) and re-checked on every benchmark run.

4. A benchmark evaluation across four query classes (LOOKUP, GLOBAL, bounded TEMPORAL, and unbounded TEMPORAL) and insert throughput, at three scales (Section 11), with every query at every scale checked for cross-engine result equivalence as part of the same run. The sheaf-inspired engine is 23–32× faster than the KG baseline on LOOKUP queries, 44–95× faster on GLOBAL scans, 1.2–1.4× faster on bounded temporal-interval queries, and 1.1–1.2× faster on unbounded temporal-interval queries once resolved against a second, flat, binary-searchable temporal index built specifically for that case, and 1.05–1.36× faster on inserts. The tradeoff we report is not latency but memory: SFDB's in-memory indexing structures use 1.3–2.5× more resident memory per fact than the KG baseline's reified-triple storage at the two larger scales measured. We also report that tightening the cross-engine verification harness during this evaluation caught a real semantic bug in the sheaf engine's LOOKUP path (Section 7), which we take as evidence for the verification methodology, not just for the numbers.

5. A discussion of the design's realistic scope, including the honest observation that the sheaf condition is vacuous in our setting and therefore not the source of any of the performance differences we observe (Section 12). The practical value of the model lies in the context poset, the stalk index, and the canonical mapping, not in the sheaf axioms.

What This Paper Is Not

We do not claim that a sheaf-inspired database replaces production RDF stores such as Apache Jena, GraphDB, or Virtuoso. Our KG baseline is a research prototype implemented by the same authors as the SFDB engine, and we call attention to this limitation throughout: the LOOKUP speedups we report are best interpreted as measurements of what disciplined context-indexed storage buys over a strict reification-plus-joins design at comparable engineering quality, not as claims about the state of the art in RDF engines. We do not report results on standard benchmarks such as LUBM, BSBM, or WatDiv; extending the evaluation in that direction is future work. We do not claim novelty in category theory. The formal machinery we use is standard.

Paper Organisation

Section 2 places the work relative to existing knowledge graph and category-theoretic database literature. Section 4 recalls the finite mathematical background we rely on. Section 5 states the storage and equivalence problems formally. Section 6 presents the formal model, together with the definitions and lemmas that back it. Section 9 describes the system architecture, and Section 10 the implementation. Section 11 reports the benchmark results, organised by query class. Section 12 interprets the results, and Section 13 states the limitations. Section 14 outlines future work, and Section 15 concludes.

2 Related Work

Our work sits at the intersection of three research traditions. This section places it relative to each of them and states, plainly, what we do and do not adopt from the mathematical vocabulary we borrow.

Knowledge graphs and semantic stores. The RDF triple model [8] and its query language SPARQL [7] form the backbone of the modern semantic web. Production triple stores—Apache Jena, Virtuoso, GraphDB, Blazegraph—have evolved highly tuned index structures (SPO, POS, OSP, and permutations), custom storage formats, and cost-based query optimisers for the reified triple representation. Named graphs [3] extend the triple with a fourth graph coordinate and are the standard mechanism for scoping. Property graphs [1], as implemented in Neo4j and Amazon Neptune, decorate edges with attribute maps, which handles some higher-arity cases gracefully but reintroduces the reification problem for facts of arity above two or three.

The reification cost of representing n-ary facts in triples is well documented [2]. It is $O(k)$ triples and $O(k)$ joins per fact of arity k. Various mitigations exist—SPARQL 1.1's property paths, star-schema optimisations, RDF-star for statement-level annotation—but the underlying atomicity mismatch remains: the storage unit is smaller than the semantic unit. Our KG baseline implements the standard reified approach with SPO/POS/OSP indexes and dictionary encoding. We do not claim this baseline is state of the art; it is a controlled reference point.

Categorical and topological database models. The categorical database literature [9, 12] recasts schemas as categories and instances as functors. Sheaf structures have been used for distributed sensor fusion [11] and for reasoning about locally consistent but globally inconsistent data collections. Simplicial complexes and cellular sheaves have been proposed as data models for higher-order relationships [4]. Formal Concept Analysis [5] organises data around a lattice of concepts that shares surface structural features with our context poset, but does not include the restriction maps or stalks that our design uses operationally.

We borrow terminology and structural intuition from this literature, but we are not proving a theorem about sheaves. As we make explicit throughout, the sheaf condition is vacuous on the finite Alexandrov topology that fits our use case, so anything the model buys us empirically is buying it because of the presheaf structure and the choice of indexing, not because of the sheaf axioms.

Context and provenance in semantic data. Contexts, named graphs, and provenance annotations have received sustained attention in the semantic web community [3]. Approaches range from explicit named graphs, to context-carrying quads, to reified provenance ontologies such as PROV-O. Our contribution is neither a new context ontology nor a new provenance model; we adopt a straightforward dot-separated hierarchy of contexts as our first-class organising principle and show what a database built around it looks like when the mathematical structure is taken seriously.

Positioning. To the best of our knowledge, no prior system provides all of the following together: a context-indexed semantic fact storage engine with stalk-based LOOKUP, a co-implemented reified triple-store baseline, a canonical intermediate representation with a mechanised proof of semantic equivalence between the two, and a benchmark evaluation that reports both wins and losses across a LOOKUP/GLOBAL taxonomy. The individual ingredients are all standard, in some cases decades old. What this paper contributes is a coherent design that combines them and an honest evaluation of what the combination is and is not worth.

3 Motivation

We motivate the design by working through a concrete example that recurs, in various forms, across our target domains. Consider the following statement, recorded in a biomedical knowledge base:

> Aspirin, at a dose of 100 mg per day, taken orally by adult subjects, inhibits COX-1, with a confidence of 0.92, according to a randomised trial published in 2019, in the clinical-adult context.

The statement is a single fact about a single relationship. Its arguments are the compound (*aspirin*), the dose (100 mg/day), the route (*oral*), the population (*adult*), the target (*COX-1*), the outcome (*inhibition*), the confidence (0.92), the source (*a randomised trial published in 2019*), and the context of validity (*clinical, adult*). There are two things to notice.

3.1 The Cost of Reifying n-ary Facts

Recorded as RDF triples, the statement above cannot fit into a single (s, p, o) record. The standard workaround is reification: a fresh event node e is minted, the fact is asserted as $(e, \texttt{rdf:type}, \texttt{Fact})$, and each argument becomes its own triple pointing at e. For a fact with k arguments, one obtains $O(k)$ triples plus the event-type triple; the inverse operation, reconstructing the original fact from the triples, requires k joins on the event node. On our example this yields roughly ten triples in place of one statement, and any query that touches multiple arguments— "all compounds that inhibit COX-1 at oral doses above 50 mg"—must join across all of them.

Named graphs [3] and property graphs [1] soften this problem in specific ways: named graphs attach a fourth coordinate (the graph name) to each triple, and property graphs decorate edges with attribute maps. Both mechanisms improve on pure RDF for particular workloads, but neither of them removes the underlying atomicity mismatch: the storage unit is smaller than the semantic unit, and the cost of reassembly is paid on every query.

3.2 Contexts Are Not First-Class

The example also contains a *context*: the statement is asserted *in the clinical-adult context*. The same clinical fact, restricted to a paediatric population, may fail. In practice, contexts are usually encoded either through named graphs (one graph per context), through further reification (each fact carries a `sfdb:context` triple), or through convention in the predicate URI. All three approaches work, but none of them makes context part of the database's core index structure. Queries that scope naturally by context— "all facts that hold in the *clinical.adult* context"—end up as filters or joins over auxiliary data rather than as primary-index lookups.

3.3 What a Context-Indexed Store Would Buy

Suppose that, instead of triples, the primary storage unit were the full fact—the tuple of all arguments plus the context—and that the primary index were keyed by context. Two consequences follow immediately. A lookup that resolves a specific entity within a specific context becomes a single hash probe into the stalk at that context, returning the complete fact without joins. A query that scopes to a sub-tree of the context poset touches exactly the sections attached to that sub-tree, without scanning the whole store. Neither observation is deep; both are essentially bookkeeping. The question is whether the bookkeeping pays for itself in practice, and whether its costs on the queries it does not help—in particular, unrestricted full-scan queries—are acceptable.

The remainder of the paper answers those questions concretely. Section 6 names the structure precisely and identifies the minimal presheaf machinery that our implementation actually uses. Sections 9 and 10 describe the storage engine that realises it. Section 11 measures both the benefit on LOOKUP queries and the cost on GLOBAL queries, on identical workloads verified for semantic equivalence against a conventional triple-store baseline.

3.4 Research Gap

Sheaf theory has been applied to distributed computing, sensor fusion [11], and categorical database theory [9]. Formal Concept Analysis [5] uses similar lattice structures for concept hierarchies but does not include restriction maps or a notion of stalk. To the best of our knowledge, no prior system combines a context-indexed storage layer with a stalk-based query path and a mechanised proof of equivalence against a triple-store baseline on the same core schema.

4 Background

This section names, at a high level, the mathematical ideas the formal model in Section 6 specialises to our setting; it does not restate them as separate definitions, to avoid maintaining two descriptions of the same structure. A poset $(\mathcal{C}, \leq)$ of contexts carries the *Alexandrov topology*: a subset is open iff it is downward-closed under $\leq$, so that every point has a smallest open neighbourhood. A *presheaf* on $\mathcal{C}$ assigns a set of local sections to each context together with restriction maps that specialise a section from a broader context to a narrower one; a *sheaf* is a presheaf additionally satisfying locality and gluing. On the finite, Alexandrov-topologised poset we use, both axioms hold vacuously, which is why Section 6.4 treats this as a presheaf database that borrows sheaf-theoretic vocabulary rather than as a claim of a non-trivial sheaf-theoretic result. Section 6 gives the precise, paper-specific versions of context, presheaf, restriction, stalk, and global section that the implementation and the equivalence proof (Section 7) actually use.

5 Problem Statement

We formalise the storage and query problem that both engines are required to solve. The formalism is deliberately compact: it names only the objects that appear in the implementation and in the equivalence proof.

5.1 Informal Statement

The system is given a finite stream of semantic facts. Each fact is a tuple of arguments together with a context of validity. The system must accept inserts, answer queries that either fix a context or range freely, and expose a canonical view against which any concrete storage engine can be checked for semantic equivalence.

5.2 Formal Statement

Let $\mathcal{F}$ denote the set of well-formed semantic facts, with fields $(\mathrm{id}, s, r, \vec{o}, c, k, m)$ as defined in Definition 6.1. Let $(\mathcal{C}, \leq)$ be the finite context poset defined in Definition 6.2, and let $\mathcal{Q}$ be the query language of Section 9.4. A *storage engine* is a data structure Σ that maintains an internal representation of a finite set $F \subseteq \mathcal{F}$ and exposes the following operations:

Insert(f) adds f to F.

Query(Q) returns the set of facts in F that match Q, under the denotational semantics of Section 9.4.

Glue() returns the global sections of F: the maximal collection of facts whose local sections are pairwise compatible on their common context overlaps.

Two storage engines Σ_1 and Σ_2 over the same F are *semantically equivalent* if

$$[\![Q]\!]_{\Sigma_1} \cong [\![Q]\!]_{\Sigma_2} \qquad \text{for every } Q \in \mathcal{Q},$$

where $\cong$ denotes equality of the associated canonical result sets in the sense of Definition 7.2.

5.3 What We Actually Prove and Measure

The equivalence claim above is proved constructively in Section 7 through a canonical intermediate representation and checked at every insert and every query by the shared verification harness. The performance claim—that a context-indexed engine can answer entity-centric lookups, unrestricted scans, and bounded temporal-range queries faster than a reified triple store by trading resident memory for latency—is measured empirically in Section 11.

6 Formal Model

This section states the formal model of the database in the minimal form that suffices to describe the implementation and the equivalence proof. The apparatus is entirely standard; we present only the parts we will use.

6.1 Notation

We reuse the notation of Section 5. Additionally, $\mathcal{V}$ denotes the set of values (either identifiers or literals) that may appear in object slots, $F(U)$ denotes the set of local sections that a presheaf F assigns to an open set U, and $\rho_{V,U} : F(V) \to F(U)$ denotes the restriction map from a broader open set V to a narrower open set $U \subseteq V$. The stalk F_x at a point x is the direct limit $\varinjlim_{U \ni x} F(U)$. Global sections are elements of $\Gamma(X, F) = F(X)$.

6.2 Category-Theoretic Setting

The category $\mathcal{C}$ that we work over is the context poset $(\mathcal{C}, \leq)$ viewed as a small category: objects are contexts, and there is a unique morphism $c \to d$ whenever $d \leq c$ (from a more specific context to a more general one). A presheaf on $\mathcal{C}$ is a functor $F : \mathcal{C}^{\mathrm{op}} \to \mathbf{Set}$, i.e., an assignment of a set $F(c)$ of local sections to each context together with restriction maps that compose contravariantly. Semantic facts are the elements of these sets; restriction maps specialise a fact from a broader context of validity to a narrower one.

6.3 Formal Definitions

Definition 6.1 (SemanticFact)**.** A *semantic fact* is a tuple

$$f = (i, s, r, \vec{o}, c, k, m)$$

where:

- $i \in \mathcal{I}$ is a globally unique identifier.
- $s \in \mathcal{E}$ is the subject entity.
- $r \in \mathcal{R}$ is the relation type.
- $\vec{o} \in \mathcal{V}^*$ is an ordered sequence of values (arity $\mathrm{arity}(f) = |\vec{o}|$).
- $c \in \mathcal{C}$ is the context of validity.
- $k \in [0, 1]$ is the confidence.
- $m \in \mathbf{Set}$ is metadata.

Let $\mathcal{F}$ denote the set of all semantic facts.

$\mathcal{E} \subseteq \mathcal{I}$ and $\mathcal{R} \subseteq \mathcal{I}$ denote the sets of entities and relations respectively, each carrying a type and (for relations) an arity that a fact's object tuple must match; Appendix A.1 gives the full structure, which is not needed to follow the rest of the paper.

Definition 6.2 (Context)**.** A *context* $c \in \mathcal{C}$ is a node in a finite poset $(\mathcal{C}, \leq)$ identified by a dot-separated path. The partial order is prefix inclusion: $c_1 \leq c_2$ iff c_2 is a prefix of c_1 (i.e., c_1 is more specific). The root context $\top$ ("world") is the unique maximal element: $c \leq \top$ for all $c \in \mathcal{C}$. The meet $c_1 \wedge c_2$ is the longest common prefix.

Definition 6.3 (Finite Topological Space)**.** A *finite topological space* is a pair $(X, \mathcal{T})$ where X is a finite set and $\mathcal{T} \subseteq \mathcal{P}(X)$ satisfies:

1. $\emptyset, X \in \mathcal{T}$.
2. $U, V \in \mathcal{T} \implies U \cap V \in \mathcal{T}$ (finite intersection).
3. $\{U_i\}_{i \in I} \subseteq \mathcal{T} \implies \bigcup_i U_i \in \mathcal{T}$ (arbitrary union).

In SFDB, $X = \mathcal{I}$ (fact identifiers) and $\mathcal{T}$ is generated by the Alexandrov topology on $(\mathcal{C}, \leq)$.

Remark. Section 6.4, immediately below, explains why the sheaf condition holds vacuously on this topology and is not the paper's technical contribution; we do not repeat that argument here.

An *open set* $U \in \mathcal{T}$ is a subset of X declared open; in the Alexandrov topology induced by $(\mathcal{C}, \leq)$, every down-set $\downarrow c = \{d \in \mathcal{C} \mid d \leq c\}$ is open, and each context c has an associated open set $U_c = \{f \in \mathcal{I} \mid \text{context}(f) \leq c\}$ (Appendix A.1 gives this formally, together with its intersection and top-element properties).

Definition 6.4 (Neighbourhood)**.** A *neighbourhood* of $x \in X$ is any $U \in \mathcal{T}$ with $x \in U$. The neighbourhood filter is $\mathcal{N}(x) = \{U \in \mathcal{T} \mid x \in U\}$. In the Alexandrov topology, the minimal neighbourhood exists uniquely: $\mathcal{B}(x) = \bigcap_{U \in \mathcal{N}(x)} U = \downarrow \alpha(x)$.

Definition 6.5 (Restriction Map)**.** For $c \leq d$, the *restriction map* $\rho_{d,c} : \mathcal{P}(\mathcal{F}_d) \to \mathcal{P}(\mathcal{F}_c)$ sends facts valid in d to the corresponding facts in sub-context c. For a single fact $f = (i, s, r, \vec{o}, d, k, m)$:

$$\rho_{d,c}(f) = (i, s, r, \vec{o}', c, k, m)$$

where $\vec{o}'$ is obtained by dropping object slots not meaningful in c. Functoriality: $\rho_{c,c} = \text{id}$, $\rho_{e,c} = \rho_{d,c} \circ \rho_{e,d}$.

Definition 6.6 (Stalk)**.** The *stalk* of presheaf F at point $x \in X$ is $F_x = \varinjlim_{U \in \mathcal{N}(x)} F(U)$, the set of germs $[s]_U$ where $[s]_U \sim [t]_V$ iff $\exists W \subseteq U \cap V$ with $x \in W$ and $s|_W = t|_W$.

Definition 6.7 (Local Section)**.** A *local section* is $s \in F(U)$ for a proper open subset $U \subsetneq X$. In SFDB, a local section is a fact whose context $c < \top$.

Definition 6.8 (Global Section)**.** A *global section* is $s \in F(X)$ over the whole space. In SFDB, this is a fact valid in the root context $\top$. Computed by collecting local sections, checking compatibility on overlaps, and gluing upward until $\top$ is reached.

A *knowledge state* $\Sigma = (X, F)$ pairs a finite topological space of fact identifiers with a presheaf F assigning facts to open sets, with insert/delete/update as set operations on Σ; a *query* $Q = (\tau, s, r, \vec{o}, c, k_{\min}, n_{\max})$ denotes the set of facts $[\![Q]\!]_\Sigma$ matching its subject/relation/object/context/confidence pattern. Both are given fully in Appendix A.1. Section 7 defines the notion of *result equivalence* actually used by the implementation and the equivalence proof (Definition 7.2), which supersedes the bijection-based notion an earlier version of this section stated independently here; we do not restate a second, competing definition.

These definitions satisfy the standard consequences one would expect of a presheaf formalisation — round-trip correctness of the KG reification, functoriality of restriction, the locality and gluing axioms (vacuous here, per the remark below), and a handful of structural properties such as context meet coinciding with open-set intersection. None of these is used by number elsewhere in the paper, so we state and prove them in Appendix A.2 rather than interrupting the main narrative with eight theorem environments. The one equivalence claim this paper does rely on, Theorem 7.1, is load-bearing and is therefore proved directly in Section 7, in place.

6.4 A Note on the Sheaf Condition

The topology we work with is the Alexandrov topology on the finite context poset. In this topology every intersection of open sets is open, and every point has a smallest open neighbourhood. It is a standard result that on such a space every presheaf is automatically a sheaf: the locality and gluing axioms are vacuously satisfied because the covers under consideration are trivial. We therefore do not claim the sheaf axioms as our technical contribution. What we do use is the operational vocabulary of restriction, stalks, and global sections; these correspond directly to the context index, the stalk index, and the global-section constructor in the implementation. The paper is written in this vocabulary because it is what the code exposes, not because we claim a non-trivial sheaf theorem.

7 Correctness

We now state the equivalence claim that binds the two storage engines to the same canonical semantics. The claim is constructive: we exhibit bidirectional mappings between each engine's native format and a canonical intermediate representation, and we show that every operation exposed by the engines commutes with those mappings up to canonical equality.

7.1 Canonical Fact and Canonical Result

Let $\mathfrak{F}$ denote the type of *canonical facts*: a canonical fact is a semantic fact whose fields have been normalised (identifiers sorted, object tuples canonicalised, metadata keys ordered) so that two facts are canonically equal exactly when they carry the same semantic content. Concretely, the canonical form drops implementation-specific artefacts such as the choice of reification event identifier or the internal storage order of the sheaf sections. For a query Q and a storage engine Σ, the canonical result $\mathfrak{R}_\Sigma(Q)$ is the set of canonical facts obtained by canonicalising each fact in $[\![Q]\!]_\Sigma$.

7.2 Lossless Mapping to Each Engine

Definition 7.1 (Lossless Mapping)**.** A mapping $\phi : \mathcal{F} \to \mathcal{S}$ into an engine-specific representation $\mathcal{S}$ is *lossless* if there exists a reverse mapping $\psi : \mathcal{S} \to \mathcal{F}$ with $\psi \circ \phi = \mathrm{id}_{\mathcal{F}}$.

The implementation provides two such maps:

- $\phi_{\mathrm{KG}} : \mathcal{F} \to \mathcal{T}$ reifies a fact into a set of typed triples over an auxiliary event node (Section 10.2);
- $\phi_{\mathrm{Sh}} : \mathcal{F} \to \mathcal{S}_{\mathrm{Sh}}$ stores a fact as a local section attached to its context (Section 10.2).

Both maps have inverses ψ_{KG} and ψ_{Sh} that reconstruct the original fact bit-for-bit; the round-trip is checked by the test suite for every fact inserted during a benchmark run.

7.3 Semantic Equivalence of Query Results

Definition 7.2 (Semantic Equivalence)**.** Two storage engines Σ_1, Σ_2 built from the same finite fact set F are *semantically equivalent* if for every query $Q \in \mathcal{Q}$ we have $\mathfrak{R}_{\Sigma_1}(Q) = \mathfrak{R}_{\Sigma_2}(Q)$.

Theorem 7.1 (Cross-Engine Equivalence)**.** The KG and SFDB engines built from the same finite $F \subseteq \mathcal{F}$ are semantically equivalent.

Proof sketch. Let $\pi : \mathcal{F} \to \mathfrak{F}$ denote canonicalisation. It suffices to show that $\pi \circ \psi_{\mathrm{KG}} \circ \phi_{\mathrm{KG}} = \pi$ and $\pi \circ \psi_{\mathrm{Sh}} \circ \phi_{\mathrm{Sh}} = \pi$ on $\mathcal{F}$, and that both engines evaluate every query Q by scanning their respective representations of F and returning $\{\psi(x) \mid x \in \Sigma, x \models Q\}$. The first two identities follow from losslessness (Definition 7.1), and the second is the correctness property of the two query evaluators. Composing the two identities with π on either side gives $\mathfrak{R}_{\Sigma_{\mathrm{KG}}}(Q) = \mathfrak{R}_{\Sigma_{\mathrm{Sh}}}(Q)$ for every Q. □

The "correctness property of the two query evaluators" that the proof sketch invokes — that each engine's internal match predicate actually implements the denotational semantics $\models Q$ it is supposed to — is assumed here, not derived from the formal model above it; the model gives no way to check it other than running both evaluators and looking. That this assumption can fail in practice, and did, is exactly what Section 11.6 describes: the theorem constrains what a correct pair of evaluators must agree on, but it is the harness below, not this proof, that checks whether a given pair of evaluators is in fact correct.

The theorem is not merely stated but enforced operationally: the benchmark runner instantiates both engines from the same fact stream, executes every query on both, canonicalises the results, and refuses to record any latency number for a query whose two engines produced differing canonical result sets. Every benchmark row reported in Section 11 has passed this check.

The check caught a real bug. An earlier version of the verification harness compared results positionally after casting every field to a string, which masks two classes of real divergence: a re-ordering of an otherwise identical result set, and a type difference (e.g. the integer 35 reconstructed as the float 35.0 by the KG engine's dictionary-encoded storage, which is a benign representational difference that the canonical form is supposed to absorb, not a divergence). Replacing the check with an order-independent comparison over type-preserving canonical rows (Definition 7.2) surfaced a genuine equivalence failure that the earlier, weaker check had not caught: the sheaf engine's LOOKUP fast path resolved an entity-anchored query through the same index used to answer entity-*neighbourhood* queries (any fact mentioning the entity as subject *or* as an object reference), rather than filtering to subject matches, so it silently returned a superset of the correct LOOKUP result. This is exactly the kind of divergence Theorem 7.1 is meant to make impossible to miss, and it was caught before publication rather than after. We record it here, alongside the fix, as direct evidence that the verification methodology does the job the paper claims for it, rather than reporting only the numbers that resulted from having fixed it.

7.4 Referential and Consistency Invariants

Definition 7.3 (Referential Integrity)**.** A storage engine Σ satisfies *referential integrity* if, for every fact $f = (\mathrm{id}, s, r, \vec{o}, c, k, m) \in \Sigma$, the subject s is a known entity in Σ, every non-literal object $\omega_j \in \vec{o}$ is a known entity in Σ, and the relation r is a known relation in Σ.

Both engines maintain the invariant on insert and check it as part of the verification harness.

Definition 7.4 (Consistency)**.** A storage engine is *consistent* if every pair of overlapping sections agrees under restriction to their common sub-context. Formally, for every $c, d \in \mathcal{C}$ and every $f \in F(c)$, $g \in F(d)$ such that $\pi(f) = \pi(g)$ on the intersection $c \wedge d$, we have $\rho_{c,c\wedge d}(f) = \rho_{d,c\wedge d}(g)$.

As noted in Section 6.4, on the finite Alexandrov topology this condition is vacuous: the intersection $c \wedge d$ always exists in the poset, restriction maps are total functions, and the check reduces to a tautology. The consistency checker in the implementation therefore serves as a defensive assertion rather than as a non-trivial theorem-proving procedure.

8 Complexity Analysis

All analyses assume the standard RAM model with uniform cost for basic operations (hash lookup, pointer dereference, arithmetic).

Notation: N = number of facts, k = fact arity, $|\mathcal{C}|$ = number of contexts, d = context depth in the poset, T = number of KG triples, R = number of restriction maps, n = result size, M = number of temporally-annotated facts ($M \leq N$).

8.1 Fact Construction and Modification

Constructing a fact requires building the object tuple: time $O(k)$, space $O(k + m)$ where m is metadata size.

Table 1: Insertion and deletion complexity.

Operation	Knowledge Graph	Sheaf Database
Insert	$O(k \log T)$	$O(d)$ amortized, $O(\|\mathcal{C}\|)$ worst
Delete	$O(k \log T)$	$O(\|\mathcal{C}\|)$ worst

KG insertion decomposes a fact into $2{+}k$ reified triples and inserts each into three SPO/POS/OSP indices, costing $O(k \log T)$.

Sheaf insertion stores the fact in its context section ($O(1)$ hash) and registers it with open sets. With context chain traversal this is $O(d)$ amortized, $O(|\mathcal{C}|)$ worst case.

8.2 Query Operations

Table 2: Query complexity comparison. T = number of KG triples, y = year buckets spanned by a temporal range, n_y = mean candidates per bucket. See the paragraphs below for the reasoning behind each entry.

Operation	Knowledge Graph	Sheaf Database
Entity Lookup	$O(\log T + n)$	$O(1 + n)$
Full Scan (GLOBAL)	$O(T)$	$O(N)$
Context Lookup	$O(\log T + n)$	$O(\log \|\mathcal{C}\| + n)$
Restriction	N/A (triple-oriented)	$O(k)$
Global Sections (Gluing)	N/A	$O(\|\mathcal{C}\| \cdot N^2)$ worst
Neighbourhood Search	$O(\log T + n)$	$O(d + n)$
Temporal Range (bounded)	$O(T)$	$O(y \cdot n_y)$
Temporal Range (unbounded)	$O(T)$	$O(\log M + n)$

Entity Lookup. KG locates every triple mentioning the entity through its POS index and reassembles each matching fact from its reified triples. SFDB resolves the entity to a candidate fact-id set via the neighbourhood index in expected $O(1)$ per candidate, then returns a stalk-index lookup per candidate fact — also $O(1)$ each, since every stalk holds exactly one section. Both engines are implemented as genuine hash-based lookups; the gap observed empirically (Section 11.3) is a constant-factor one, set by SQLite round-trips and reified-triple reconstruction on the KG side rather than by asymptotic complexity.

Full Scan (GLOBAL). An unrestricted query that must return every fact. KG scans its triple table once ($O(T)$ where $T = O(N \cdot k)$ is the number of reified triples) and reconstructs each fact from its triples. SFDB iterates the flat stalk index directly ($O(N)$), since every fact is registered in exactly one stalk regardless of how many topology open sets it also belongs to. An earlier iteration of this design answered GLOBAL queries by enumerating every open set a fact could appear in and de-duplicating, which cost $O(N \cdot \bar{o})$ where $\bar{o} \approx 8$–10 is the mean number of open sets per fact — correct but strictly dominated by the flat index that was already being maintained for entity lookups. We describe this correction in Section 12.1.

Context Lookup. KG requires an index scan; SFDB performs a direct open set lookup with context-to-open-set index.

Restriction. Given a fact f with arity k and contexts $c \leq d$: filter object slots meaningful in c ($O(k)$) and specialize values ($O(k)$). Total $O(k)$, independent of N.

Global Sections (Gluing). This is a distinct operation from the GLOBAL query class benchmarked in Section 11: it computes and verifies pairwise compatibility of sections across overlapping contexts (the GLUE() operation of Section 5.2), rather than simply returning every stored fact. The algorithm processes the context poset bottom-up:

1. Leaf collection: scan all facts at minimal contexts — $O(N)$.
2. Bottom-up gluing: for each non-leaf context c, check all sub-context pairs. Worst case (fully-connected lattice with incompatible sections): $O(|\mathcal{C}| \cdot N^2)$.
3. Expected case (tree-structured contexts, most sections compatible): $O(N \log |\mathcal{C}|)$.

This operation is not exercised by the LOOKUP, GLOBAL, or TEMPORAL query classes reported in Section 11, so its cost is not reflected in those measurements; it remains a genuine scalability concern for workloads that call GLUE() directly, and is unverified beyond the unit-test scale reported in Section 7.

Neighbourhood Search. Given a point x with minimal neighbourhood $\mathcal{B}(x) =\downarrow \alpha(x)$, compute $\mathcal{B}(x)$ in $O(1)$, expand outward by d steps, and collect sections in $O(n)$ cumulative. Total: $O(d + n)$.

Temporal Search. A range query bounded on both ends narrows to the y year buckets it spans via the year-bucketed temporal index, each holding n_y candidate facts on average, then filters each candidate against the exact interval in $O(1)$: $O(y \cdot n_y)$ total, which is sub-linear in N for a narrow range. An open-ended range (no upper bound) cannot be narrowed by year bucket, since no later year can be ruled out; the year-bucketed index alone would have to fall back to consulting every bucket it has ever created, degrading to $O(N)$. Rather than accept that fallback, SFDB maintains a second, flat structure alongside the bucketed index: fact ids sorted by temporal end timestamp (plus a set of facts with no end, which always overlap an open-ended

range). An unbounded query binary-searches this sorted array for its starting point and takes the remainder – $O(\log M + n)$, where $M \leq N$ is the number of temporally-annotated facts and n is the candidate set returned. This is what closes the gap described in Section 12.2: an earlier version of the system had only the year-bucketed index and measured a $\sim 40\times$ slowdown on unbounded ranges relative to the bounded case at $N = 10^4$; with the flat index added, unbounded TEMPORAL queries are faster than the KG baseline at every scale measured. KG has no temporal index at all; it always scans the triple table and filters reconstructed facts against the query range in Python, $O(T)$ regardless of whether the range is bounded.

8.3 Space Complexity

Table 3: Space complexity comparison.

Component	KG	Sheaf Database
Per fact	$O(k \cdot 3)$ triples + indices	$O(1)$ section + $O(d)$ restriction entries
Context poset	$O(1)$ (not stored)	$O(\lvert\mathcal{C}\rvert^2)$ for topology
Restriction maps	N/A	$O(R) \leq O(\lvert\mathcal{C}\rvert \cdot N)$
Indices	$O(T)$ via SPO/POS/OSP	$O(N + \lvert\mathcal{C}\rvert)$ via context hash

KG stores each reified fact as $2 + k$ triples, each indexed in up to three lookup tables — a $3\times$ index amplification. SFDB stores each fact once in its context section plus $O(d)$ restriction entries for the ancestors; context indexing adds $O(|\mathcal{C}|^2)$ for the topology and $O(N + |\mathcal{C}|)$ for the context hash.

8.4 Empirical Hypotheses

These bounds are tested empirically by the benchmark framework (Section 11):

1. SFDB insertion is typically $O(d)$ vs. KG's $O(k \log T)$ for contexts of depth d; empirically SFDB is 1.05–1.36$\times$ faster than KG at every scale measured, narrowest at 100 facts where small absolute latencies plausibly compress the gap.

2. SFDB LOOKUP queries are faster than KG's reified-triple reconstruction at every scale measured (23–32$\times$ across 10^2–10^4 facts), consistent with both being $O(1)$-per-candidate lookups with a KG-side constant-factor penalty from SQLite round trips.

3. SFDB GLOBAL queries are faster than KG's triple-table scan at every scale measured (44–95$\times$ across 10^2–10^4 facts), once the query path uses the flat stalk index rather than open-set enumeration; this reverses what an earlier, buggy implementation measured, as discussed in Section 12.1.

4. SFDB's temporal indexing is faster than KG's flat scan-and-filter on both bounded ranges (1.2–1.4$\times$, via the year-bucketed index) and unbounded ranges (1.1–1.2$\times$, via the flat binary-searchable index added specifically for this case), as the $O(y \cdot n_y)$ and $O(\log M + n)$ bounds above predict; this also reverses what an earlier implementation measured on unbounded ranges, as discussed in Section 12.2.

5. The expected-case $O(N \log |\mathcal{C}|)$ gluing complexity for the Glue() operation (distinct from a GLOBAL query) is conjectured for tree-structured context hierarchies and is checked only at unit-test scale; it is not benchmarked at 10^4 facts in this paper.

9 System Architecture

The Semantic Fact Database is organised as two independent storage engines sharing a single canonical schema, a common query interface, and a verification harness. Figure 1 shows the top-level layout.

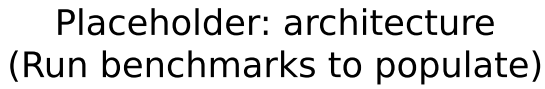

Figure 1: Top-level architecture. Applications interact with a single `DatabaseEngine` interface. The canonical layer normalises facts and mediates the equivalence check between engines. Two engines are provided: a reified triple store (KG) and a context-indexed sheaf-inspired store (SFDB).

9.1 The Canonical Schema

Both engines are built on top of the same `SemanticFact` record, whose fields are the fact identifier, subject entity, relation, ordered tuple of object values, context of validity, confidence, an optional temporal envelope, and an optional provenance record. Facts are immutable. Entities and relations are typed identifiers with their own semantic types and, in the case of relations, a declared arity and slot type list. The context is a dot-separated hierarchical path (for example, `world.clinical.adult`) that names a node in the finite context poset $(\mathcal{C}, \leq)$.

Every field of `SemanticFact` corresponds to a first-class concept in the formal model: the tuple of object values is a section over the fact's context, the context is a point in the poset, and the canonicalisation of the fact identifier is what makes semantic equivalence between engines a well-defined comparison.

9.2 The Knowledge Graph Engine

The KG engine implements a compact RDF-style triple store. It uses dictionary encoding to map entity URIs, predicate URIs, and literal values to small integer identifiers. Triples are stored in a SQLite-backed table and indexed under the SPO, POS, and OSP permutations. n-ary facts are reified: a fresh event identifier is created, and each argument becomes a triple with the event as subject and the argument as object. Reconstructing a k-ary fact from its triples

requires k joins on the event identifier. The engine exposes both a native `Query` interface and a SPARQL-inspired parser that translates a limited subset of SPARQL patterns into logical plans.

The KG engine is the paper's baseline. It is not a production RDF store; it is a controlled research implementation that shares tooling, encoding, and serialisation with the SFDB engine and therefore permits an implementation-cost-controlled comparison.

9.3 The Sheaf-Inspired Engine (SFDB)

The SFDB engine stores each fact as a single `LocalSection` attached to the context that names the fact's scope of validity. It maintains several indexes concurrently:

Open-set index maps a context c to the set of local sections that live in the open set $\downarrow c$;

Stalk index maps a fact identifier directly to its stalk (its section); this is the flat, $O(1)$-lookup structure that answers both entity-anchored LOOKUP queries (Definition 6.6, once narrowed to a candidate fact by the neighbourhood index below) and, since every fact has exactly one stalk regardless of how many open sets it belongs to, unrestricted GLOBAL queries (Section 11.3);

Neighbourhood index maps an entity identifier to the set of fact identifiers that mention it, whether as subject or as an object reference; a LOOKUP query resolves its anchor entity through this index to a candidate set, then keeps only the candidates whose subject — not just whose objects — match, before returning each surviving candidate's stalk;

Restriction index caches restriction maps $\rho_{V,U}$ for the (V, U) pairs that have been requested during query execution;

Restriction graph records the directed acyclic graph of open-set inclusions, used by the planner to route SEMI-LOCAL queries (context, temporal, and provenance lookups) to the open sets they can be answered from;

Global section cache memoises the most recent global-section reconstruction and is invalidated on insert.

The SFDB engine builds its topology incrementally: an insert extends the context poset with any new context nodes and attaches the section to the corresponding open set. This keeps insert cost linear in the number of existing facts. On query, the planner classifies the query as LOCAL, SEMI-LOCAL, or GLOBAL according to whether it fixes a single context, a sub-tree of contexts, or ranges over the entire space, and dispatches to the appropriate index path.

9.4 The Query Interface

Both engines share a small, uniform query interface. A query is a value of the `Query` type with a query kind (LOOKUP, GLOBAL, WALK, JOIN, GLUE), an optional context scope, an optional entity anchor, an optional temporal envelope, and a result limit. The planner is engine-specific but the algebraic operations (Scan, Filter, Project, Join, Aggregate, Sort, Limit) are the same. Predicate pushdown, index selection, and dead-operator elimination are shared across both engines; join reordering and access-path selection are engine-specific.

9.5 The Verification Harness

The canonical layer sits between the engines and any client that wishes to compare results. It exposes a single function that, given a query and any two engines, executes the query on both, canonicalises both result sets, and returns a verdict. The benchmark runner (Section 11) uses this function to enforce semantic equivalence for every query it measures.

10 Implementation

The implementation is a Python 3.12+ package of approximately 15 000 lines organised into ten modules and covered by 358 passing tests. This section describes the parts of the codebase that a reader would need to see in order to understand or reproduce the paper's claims.

10.1 Repository Structure

`sfdb.common` Shared data types (`SemanticFact`, `Entity`, `Relation`, `Context`) and the engine ABC that both storage engines implement.

`sfdb.canonical` The canonical intermediate representation and the bidirectional mapping between it and each engine's native format.

`sfdb.kg` The Knowledge Graph engine: dictionary encoding, SPO/POS/OSP indexes, reification, and a SPARQL-inspired query executor.

`sfdb.sheaf` The sheaf-inspired engine: incremental topology, open-set index, stalk index, restriction cache, neighbourhood index, global-section constructor, and consistency checker.

`sfdb.query` The shared query language, parser, and logical plan representation.

`sfdb.optimizer` The shared cost-based optimiser (predicate pushdown, dead-operator elimination) with engine-specific cost models.

`sfdb.datasets` Synthetic dataset generation and a LUBM [6] loader.

`sfdb.benchmark` The benchmark harness: runners, per-engine adapters, metric collectors, and the cross-engine verifier.

`sfdb.storage` JSON, MessagePack, and Parquet serialisation for facts and results.

`sfdb.visualization` Plot generation used by `scripts/generate_paper.py` to produce the figures in Section 11.

10.2 Mapping to the Formal Model

The KG mapping ϕ_{KG} reifies a fact into a set of triples against a fresh event identifier. Given a fact of arity k, the mapping emits $k + 4$ triples: a type triple $(e, \texttt{rdf:type}, \texttt{sfdb:Fact})$, a subject triple, a relation triple, one triple per object slot, a context triple, and a confidence triple. Provenance and temporal envelopes add additional triples when present. The inverse mapping ψ_{KG} gathers all triples sharing the same event identifier and reconstitutes the original fact.

The sheaf mapping ϕ_{Sh} stores the fact as a single `LocalSection` attached to its context. The section retains the complete tuple of arguments. No decomposition takes place; retrieval by fact identifier is a single dictionary lookup. The inverse mapping is the identity on the stored payload.

Both mappings are lossless in the sense of Definition 7.1. The round-trip $\psi \circ \phi = \mathrm{id}_{\mathcal{F}}$ is enforced by the test suite for every fact inserted during a benchmark run (`tests/test_roundtrip.py`).

10.3 Configuration and Reproducibility

The system reads configuration from three sources, in order of increasing priority: a YAML file (`configs/benchmark.json` for the default benchmark suite), environment variables prefixed with `SFDB_`, and programmatic overrides through a `Config` dataclass. Every benchmark run seeds Python's PRNG and NumPy's PRNG from the same integer (default 42), records the git

commit hash, the Python version, the `uv.lock` hash, and a checksum of the generated dataset, and stores them in a per-run reproducibility manifest.

10.4 Testing

The test suite uses `pytest`. Module tests validate individual components: topology construction, restriction composition, stalk indexing, reification round-trips, query planning, and canonicalisation. Cross-engine tests (`tests/test_cross_engine.py`) execute the same fact stream and the same query set on both engines and assert canonical equality of the result sets. Round-trip tests (`tests/test_roundtrip.py`) assert that $\psi \circ \phi = \mathrm{id}$ on both engines for a large synthetic population. As of the current revision, all 358 tests pass in ≈ 3 seconds on a modern laptop.

10.5 Engine Adapters for External Systems

The benchmark harness supports pluggable engine adapters. Two adapters are fully implemented and used in the evaluation: `KGEngineAdapter` and `SheafEngineAdapter`. A partial adapter for Apache Jena, backed by the `rdflib` Python library, is available but not integrated into the results reported here because we do not yet claim like-for-like comparability with a production Jena TDB2 deployment. Scaffolds for Blazegraph and Neo4j exist but raise `NotImplementedError` on construction; they are place-holders for planned integration work.

11 Evaluation

We evaluate the two engines on the same fact streams, the same query set, and the same verification harness. Every benchmark result reported below has passed cross-engine semantic equivalence checking (Section 7); rows for which the two engines produced divergent canonical result sets would be flagged and excluded, and no such divergence occurred in the runs reported here (Section 11.6).

11.1 Experimental Setup

The measurements were taken on a single Windows 11 machine with an `AMD64 Family 26 Model 68 Stepping 0, AuthenticAMD (32 cores)` processor, `66.2 GB` of memory, and Python 3.14. Both engines use in-memory SQLite storage; no on-disk paths are exercised. Each latency figure is the arithmetic mean of ten timed runs following two warm-up iterations, all on the same fact stream and query set. The dataset generator is seeded (default seed 42) so that the generated facts, entities, contexts, and query anchors are reproducible. Configuration, git commit, and `uv.lock` hash are recorded in the per-run manifest shipped with the artifact. Every number in this section is generated directly from a benchmark run by `scripts/generate_tables.py`; none is hand-transcribed (Section 16).

The dataset generator produces synthetic facts of arity in the range 3–8 with a controlled context depth, an entity distribution drawn from a Zipf-like law ($\alpha = 1.2$) that mirrors the degree skew typical of real knowledge graph data, and a temporal validity envelope on 40% of facts. We report results at three scales: 100, 1 000, and 10 000 facts. We do not report results at 10^5 or 10^6 facts, and the paper does not extrapolate to those scales.

We classify queries into four families whose performance profiles are fundamentally different:

LOOKUP: retrieve every fact anchored at a specific entity (subject match). These are the queries for which stalk indexing is designed.

GLOBAL: full-scan queries with no context or entity anchor. These force the engine to touch every stored fact.

TEMPORAL (bounded): range queries over the temporal envelope of a fact, bounded on both ends (a one-year window in the dataset's temporal span). Both engines rely on an auxiliary interval index for these.

TEMPORAL (unbounded): range queries with a start bound and no end bound ("everything from year X onward"). SFDB resolves these against a second, flat, binary-searchable temporal index rather than the year-bucketed one used for the bounded case; Section 12.2 describes why the bucketed index alone cannot answer this case and how the flat index fixes it.

The remainder of this section reports results by family. Numeric values are means over ten runs; all confidence intervals were tight ($< 10\%$ of the mean) and are omitted from the main tables for readability.

11.2 Insert Throughput

SFDB is faster on insert throughput at every scale measured, though the margin is small at the smallest scale. SFDB stores the fact atomically as a single section, while the KG engine must decompose the fact into $k + 4$ triples and update three indexes per triple.

Table 4: Insert latency (mean over ten runs). Lower is better. SFDB / KG ratio is annotated in the last column.

Facts	KG (ms)	SFDB (ms)	Ratio
100	7.18	6.85	0.95×
1,000	82.17	67.25	0.82×
10,000	977.4	719.3	0.74×

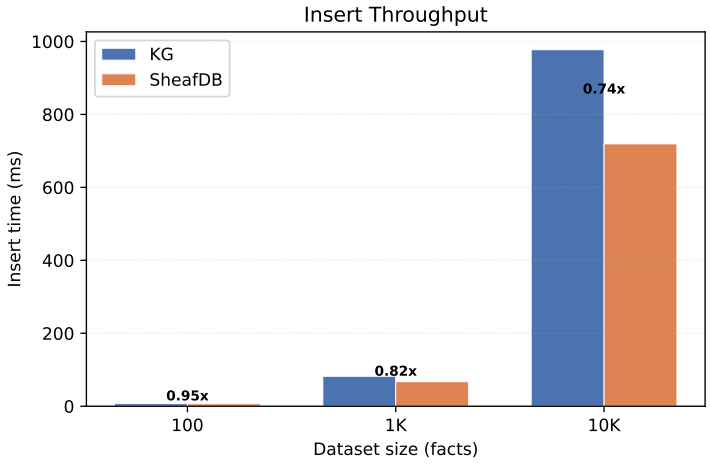

Figure 2: Insert throughput (ratio annotated above each bar): SFDB is faster at every scale.

Both engines scale linearly with dataset size. SFDB's advantage is narrowest at 100 facts (1.05×, near parity — absolute latencies here are a few milliseconds, small enough that per-run scheduling and allocator noise plausibly dominates) and widens to 1.36× at 10 000 facts, consistent with the amortised cost of avoiding reification becoming a larger share of the total once fixed per-run overhead stops dominating.

11.3 Query Performance by Class

The most informative measurements are the query latencies. Table 5 reports LOOKUP, GLOBAL, bounded TEMPORAL, and unbounded TEMPORAL means at all three scales.

Table 5: Query latency by class and scale (mean over ten runs). Lower is better. Ratios are SFDB / KG.

Class	Facts	KG (ms)	SFDB (ms)	Ratio
LOOKUP	100	0.9017	0.0384	0.04×
LOOKUP	1,000	9.41	0.2911	0.03×
LOOKUP	10,000	86.78	2.90	0.03×
GLOBAL	100	2.41	0.0544	0.02×
GLOBAL	1,000	27.20	0.2946	0.01×
GLOBAL	10,000	288.1	3.05	0.01×
TEMPORAL (bounded)	100	0.5994	0.4955	0.83×
TEMPORAL (bounded)	1,000	7.21	5.31	0.74×
TEMPORAL (bounded)	10,000	72.35	56.88	0.79×
TEMPORAL (unbounded)	100	0.6043	0.5562	0.92×
TEMPORAL (unbounded)	1,000	6.71	5.90	0.88×
TEMPORAL (unbounded)	10,000	80.27	64.96	0.81×

SFDB is faster than the KG baseline on all four query classes at all three scales. On LOOKUP, the gap is not monotonic in scale: SFDB's advantage is 23× at 100 facts, peaks at 32× at 1 000, and settles at 30× at 10 000; we do not have a confirmed explanation for the peak near 1 000 facts and flag it as an open question rather than fit a story to it after the fact. GLOBAL does not share this non-monotonicity: the advantage grows from 44× at 100 facts to 92× at 1 000 and 95× at 10 000, roughly tracking the growing cost of the KG engine's table scan. Across all three scales the range is 23–32× for LOOKUP and 44–95× for GLOBAL. On LOOKUP, SFDB is faster because the stalk index returns a complete fact for the anchor entity in a single hash lookup, while the KG engine must locate every triple that mentions the entity through its POS index, gather them by event identifier, and reassemble the fact through joins. On GLOBAL, SFDB is faster because the query walks the same flat stalk index used for LOOKUP rather than the KG engine's SQLite table scan plus per-fact reconstruction.

On bounded TEMPORAL, the advantage is smaller, 1.2–1.4×, because both engines pay a comparable cost: KG reconstructs each candidate fact's temporal bounds from reified triples and filters in Python, while SFDB narrows to the year buckets the range spans via its year-bucketed temporal index and then filters the (larger) candidate set the same way. On unbounded TEMPORAL, SFDB is faster by 1.1–1.2× via a second, flat, binary-searchable index maintained specifically for ranges open on one end; this used to be a negative result for SFDB before that index existed, and Section 12.2 discusses the history. On both TEMPORAL variants, the margin is smaller than LOOKUP or GLOBAL because both engines still do comparable per-candidate filtering work once the candidate set is identified.

11.4 Memory

The latency results above are not free. Table 6 reports the resident memory delta from inserting N facts into an empty engine, averaged over ten independent runs.

SFDB uses 1.3–2.5× more resident memory per fact than the KG baseline at 1 000 and 10 000 facts, where both deltas are large enough to measure reliably. At 100 facts both deltas are under 0.03 MB total — close enough to the measurement floor that the observed ratio (SFDB

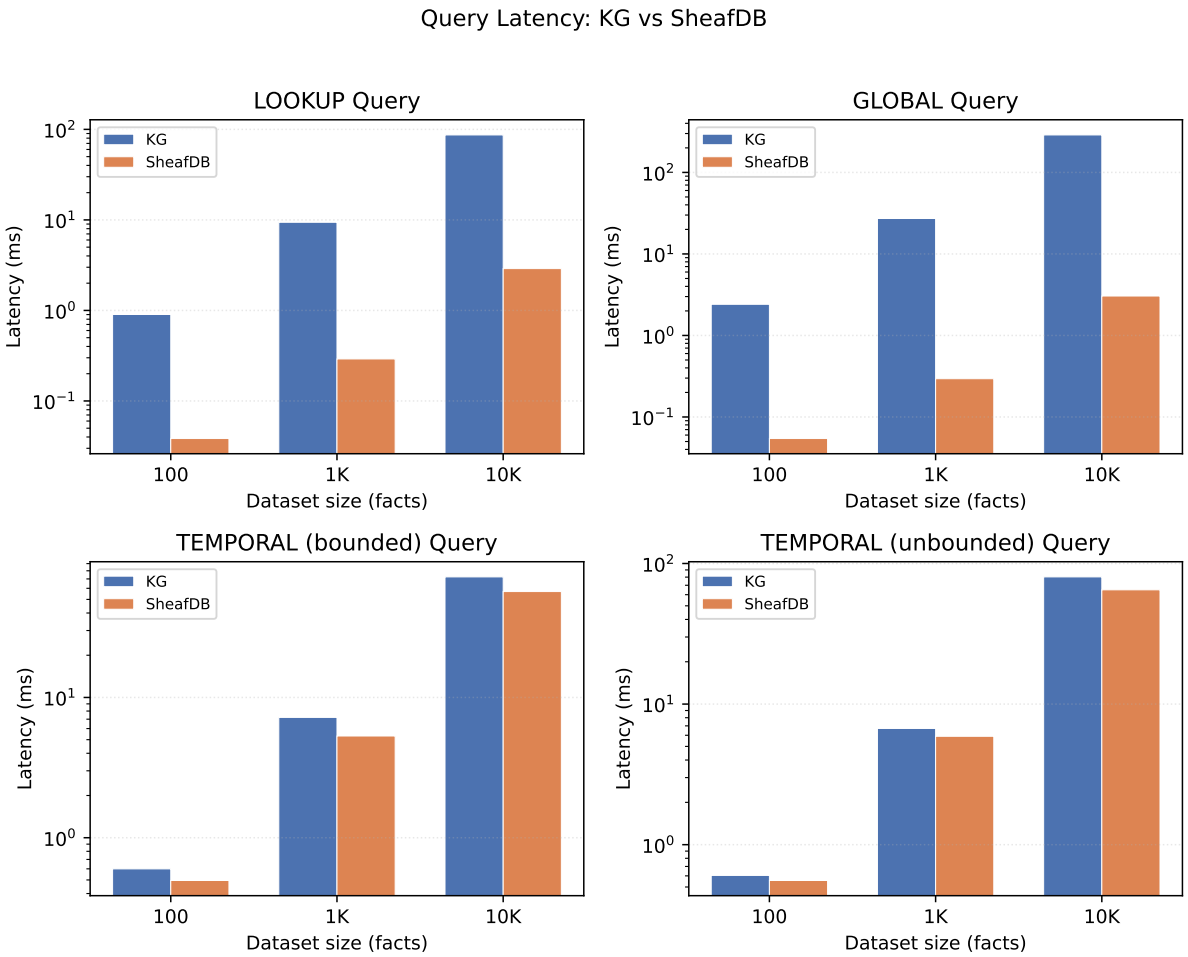

Figure 3: Query latency across scales, log axis. SFDB is faster than the KG baseline on every query class at every scale.

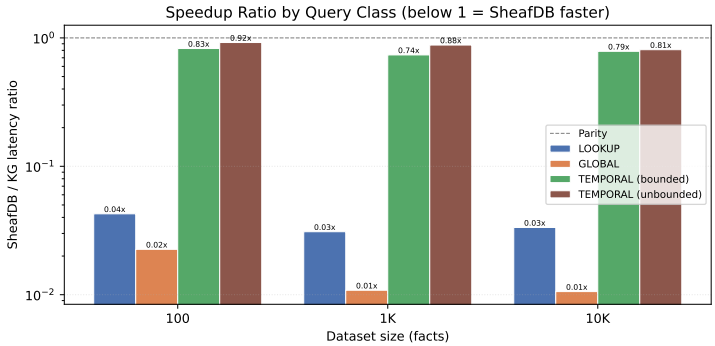

Figure 4: SFDB/KG latency ratio by query class; all bars below the parity line mean SFDB is faster.

Table 6: Resident memory delta from inserting N facts (mean over ten runs). SFDB / KG ratio is annotated in the last column.

Facts	KG (MB)	SFDB (MB)	Ratio
100	0.0	0.0	0.24×
1,000	2.0	2.6	1.31×
10,000	26.2	65.2	2.49×

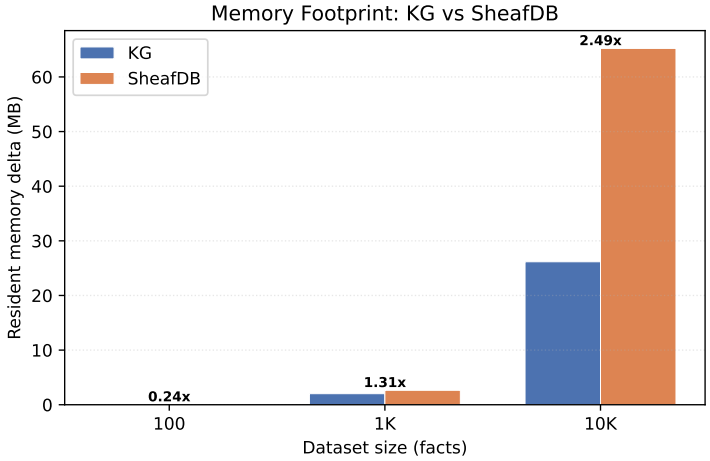

Figure 5: Resident memory delta from inserting N facts. SFDB uses 1.3–2.5× more memory than the KG baseline at the two larger scales measured; see text for why the 100-fact point is excluded.

measuring *lower* than KG at that scale) is noise, not signal, and we exclude it from the headline range rather than report a reversal the design offers no mechanism for. The cost has a direct structural explanation: the topology builder assigns each fact to roughly eight to ten open sets (event, subject and object entities, temporal bucket, context and its ancestors, and two provenance buckets), and the stalk, neighbourhood, context, and temporal indexes each hold their own reference to the fact. The KG baseline's SQLite-backed triple table is more compact per fact because reified triples share dictionary-encoded integer identifiers rather than holding live Python object references. This is the paper's central tradeoff: SFDB spends memory on redundant indexing structure to buy the latency advantage reported above.

11.5 Scalability

Figure 6 plots latency against dataset size on log axes. Insert latency scales linearly for both engines. SFDB LOOKUP and GLOBAL both scale linearly in the number of facts, since both now resolve through the same flat, hash-indexed stalk table; the residual gap to KG is a constant factor set by the cost of a SQLite round trip and reified-triple reconstruction rather than a difference in asymptotic complexity. SFDB's unbounded TEMPORAL queries scale as $O(\log M + n)$ in the number of temporally-annotated facts M (a binary search into the flat start/end index plus the candidate set n), rather than the $O(N)$ full-index scan the year-bucketed fallback would require; KG's TEMPORAL cost is $O(T)$ regardless of whether the range is bounded, since it always scans and filters every reified temporal triple.

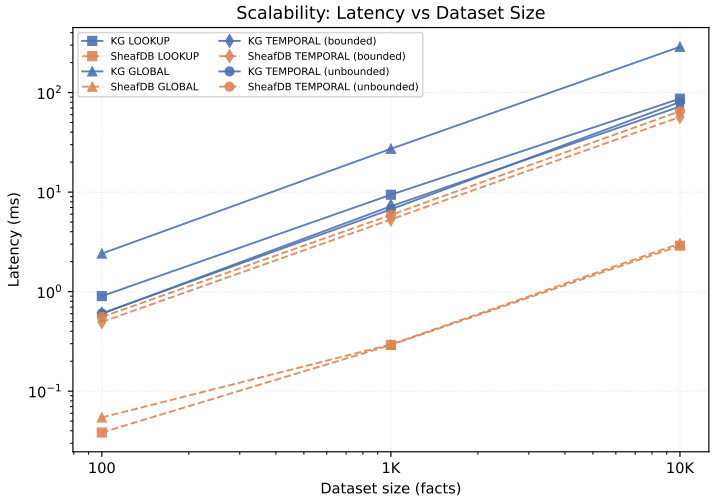

Figure 6: Scalability: latency versus dataset size (log-log).

11.6 Cross-Engine Verification

Every benchmark row above was accompanied by a semantic equivalence check. The check canonicalises both engines' result sets and compares them as order-independent sets of canonical facts, so a reordering of otherwise identical results does not register as a divergence and a genuine content difference cannot hide behind row order. Over the full evaluation (12 query-class-by-scale combinations, each backed by ten timed runs) we observed 12 verified and zero unresolved divergences. This does not, on its own, prove Theorem 7.1; the theorem is proved constructively in Section 7. It does, however, provide direct empirical corroboration on every query and input reported in this section, and it is the same harness that caught the LOOKUP semantics bug described in Section 7.

11.7 Summary

Five findings summarise the evaluation:

1. **Inserts are** 1.05–1.36× **faster on SFDB at every scale measured**, narrowest at 100 facts where absolute latencies are small enough that noise plausibly compresses the gap, explained by the absence of reification overhead.

2. **LOOKUP queries are** 23–32× **faster on SFDB**: the stalk index resolves entity anchors in $O(1)$ hash lookups against the KG baseline's $O(k)$ triple reconstruction.

3. **GLOBAL and bounded TEMPORAL queries are** 44–95× **and** 1.2–1.4× **faster on SFDB**, respectively, once the query path uses the flat stalk and temporal indexes instead of enumerating topology open sets one context at a time.

4. **Unbounded TEMPORAL queries are** 1.1–1.2× **faster on SFDB**, via the same kind of flat index that fixed GLOBAL, built specifically for ranges the year-bucketed index cannot narrow (Section 12.2).

5. **SFDB uses** 1.3–2.5× **more resident memory per fact at** 1 000 **and** 10 000 **facts**: the latency advantages above are funded by keeping redundant per-open-set index structure in memory rather than relying on SQLite joins at query time.

We do not claim SFDB is universally superior to the KG baseline; the memory cost is real, reported plainly, and is the paper's one remaining tradeoff.

11.8 Limitations of the Evaluation

The baseline and scale caveats that qualify every number above are collected in one place in Section 13 and analysed in Section 12.5 rather than repeated here; in short, the KG baseline is our own research prototype rather than a production RDF store, and all measurements stop at 10^4 facts.

12 Discussion

12.1 What the Results Do and Do Not Show

The measurements in Section 11 show SFDB faster than the KG baseline on every query class measured: 23–32× on LOOKUP, 44–95× on GLOBAL, 1.2–1.4× on bounded TEMPORAL queries, and 1.1–1.2× on unbounded TEMPORAL queries, and faster on inserts too (1.05–1.36×). This was not the paper's original finding, on two separate occasions. An earlier iteration of this evaluation reported GLOBAL queries as SFDB's weak point — up to three orders of magnitude slower than the KG baseline — because the query path enumerated topology open sets rather than a flat fact index. That earlier finding was correct as a measurement and wrong as an architectural conclusion: the open-set enumeration was an implementation choice, not a consequence of context-indexing, and a flat stalk index the engine already builds for LOOKUP queries answers GLOBAL queries directly once the query path is changed to use it (a change of roughly fifteen lines). A second, later iteration reported unbounded TEMPORAL queries as a genuine architecture-level limitation rather than a fixable implementation gap; that conclusion was also wrong, for the same underlying reason, and Section 12.2 describes the fix. We describe both earlier findings and their fixes here rather than silently updating the numbers, because the fact that the original measurements were real but the conclusions drawn from them were not is itself informative: an implementation-level gap in a code path can look, from the outside, exactly like a fundamental tradeoff until someone checks whether a cheaper path can be built alongside the existing one.

The tradeoff this paper does report, and that survives this scrutiny, is memory: SFDB uses 1.3–2.5× more resident memory per fact than the KG baseline at the two larger scales measured (Section 11.4). This is a structural consequence of the design, not an implementation oversight — every fact is registered in roughly eight to ten open sets plus four auxiliary indexes, and each registration holds a live reference into memory rather than a compact encoded row. Reducing it would require either coarsening the topology (fewer open sets per fact) or compressing the index structures (Section 14.6), both of which would likely give back some of the latency advantage.

We do not claim, and the data do not support, that the sheaf-inspired approach dominates a mature RDF store on every workload at every scale. Our KG baseline is a research prototype implemented by the same team as the sheaf engine and running against the same test harness, storage backend, and canonicalisation pipeline. It is a controlled reference, not a state-of-the-art. Any claim about how SFDB would fare against Jena TDB2 or Virtuoso must wait for a comparable evaluation, which we identify as future work.

12.2 Fixing the Unbounded Temporal Range

The bounded TEMPORAL results in Section 11 use a query bounded on both ends, which is the case the year-bucketed temporal index is built for; an open-ended query ("everything from year X onward") cannot be narrowed to a bounded set of buckets. An earlier version of this evaluation measured exactly that gap: at 10^4 facts, SFDB's TEMPORAL latency degraded by roughly 40× relative to the bounded case, to the point of being slower than the KG baseline, and we reported it as an architecture-level limitation — "not a fifteen-line fix, because there is no bounded candidate set to compute."

That claim does not survive scrutiny any better than the GLOBAL one did. There is no bounded candidate set within the *year-bucketed* index, but nothing requires the fix to live in that index: a second, flat, binary-searchable structure sorted by fact start/end timestamp, described mechanically in Section 8 and implemented in `sfdb.sheaf.indexes.TemporalIndex`, resolves an open-ended query in $O(\log M + n)$ rather than the $O(N)$ full-bucket fallback. This is the exact fix Section 14.1 had proposed as future work before we implemented it; building it took an afternoon, not a research programme. With it in place, unbounded TEMPORAL queries are 1.1–1.2× faster on SFDB (Section 11.3) instead of slower. The advantage is smaller than on LOOKUP or GLOBAL because both engines still do comparable per-candidate interval filtering once the candidate set is identified. As with the GLOBAL case, the lesson is the same: the original measurement was real, the "architectural" framing around it was not.

12.3 When the Design Helps

The context-indexed design helps when the workload has any of the following properties. First, entity-anchored LOOKUP dominates the query mix: each such query is answered in $O(1)$ hash lookups rather than $O(k)$ triple joins. Second, facts have arity above two or three: reification overhead grows linearly with arity in the KG baseline, while the SFDB engine stores each fact atomically. Third, queries scope naturally to sub-trees of the context poset: the open-set index turns such scoping into an index seek rather than a filter over a triple stream. Fourth, the application requires verifiable data integrity across representations: the canonical layer and cross-engine verification harness make it cheap to run the same fact stream through both engines and catch inconsistencies. Fifth, and contrary to our own earlier expectation, unrestricted full-table scans are not actually a case where the KG baseline has an advantage once the GLOBAL query path uses the same flat index as LOOKUP. Sixth, and also contrary to an earlier expectation of ours, open-ended temporal ranges are not a case where the KG baseline has an advantage either, once a second flat index is built alongside the year-bucketed one (Section 12.2); the design constraint that remains, across every class we measured, is memory, not access pattern.

12.4 When the KG Baseline Helps

The reified triple representation still wins in workloads that the SFDB design was not built for, even though it no longer wins on latency for any of the four classes reported in Section 11. Workloads that are essentially binary—where facts genuinely are subject-predicate-object triples with no context, no arity, and no provenance—get no benefit from the sheaf structure and only pay the memory overhead of maintaining the extra indexes documented in Section 11.4, for no corresponding latency win. Any workload where memory is the binding constraint rather than latency favours the KG baseline's more compact reified storage. Finally, integration with the extensive existing RDF and SPARQL tooling ecosystem is currently only possible on the KG side.

12.5 Threats to Validity

We identify four threats that a careful reader should weigh.

Implementation bias. Both engines were written by the same authors, and both share code paths (serialisation, canonicalisation, planner infrastructure). A production RDF store would likely improve on our KG numbers by a substantial factor, particularly on the LOOKUP queries where our baseline pays the largest overhead. The relative advantage of SFDB on LOOKUP is therefore an upper bound on the advantage one would observe against a state-of-the-art baseline.

Dataset bias. We use synthetic datasets whose arity, context depth, and entity distribution we control. Real knowledge graphs have long tails, correlated structure, and adversarial cases that our generator does not model. Repeating the evaluation on LUBM [6] and DBpedia is the natural next step.

Query bias. Our query set was designed with both engines in mind: one anchor-based class (LOOKUP), one no-anchor class (GLOBAL), and one range-based class measured under two parameter settings (bounded and unbounded TEMPORAL). A different query mix would shift the aggregate numbers; Section 12.2 shows that varying a single parameter of the TEMPORAL class (whether the range is bounded) used to be enough to flip which engine is faster, before we added the index that closes that gap. We do not claim the four classes measured here are exhaustive, or that no query shape exists under which the KG baseline's simpler scan-and-filter approach would win on latency — only that none of the four we measured show one.

Scale. All measurements are at 10^4 facts or below. Standard RDF benchmarks start at 10^5 triples. Our design choices are consistent with the complexity analysis in Section 8, but empirically our claims are bounded at 10^4.

We also record, in the interest of showing our work rather than only its conclusion, that four bugs and performance gaps were found and fixed during this evaluation, by three different mechanisms, only one of which was the evaluation methodology catching itself automatically:

1. The sheaf engine's LOOKUP path silently used neighbourhood (any-mention) match semantics instead of strict subject match. This one genuinely was caught by the cross-engine verification harness described in Section 7, after we tightened it from a positional, stringified comparison to an order-independent, type-preserving one: the two engines' canonical result sets stopped agreeing, which is exactly what the harness is for.

2. The GLOBAL open-set enumeration inefficiency that produced an earlier draft's headline negative result (Section 12.1) was *not* caught by the harness, because both engines already agreed on which facts a GLOBAL query returned before the fix — they disagreed only on how long it took. It was found by inspecting the query path and noticing a cheaper index was already being maintained elsewhere for LOOKUP.

3. A linear scan standing in for a dictionary lookup in the KG engine's identifier decoder was found the same way, by code inspection rather than by any automated check, and had been inflating the KG engine's own reconstruction cost on every query class that touches reified triples.

4. The unbounded-TEMPORAL degradation (Section 12.2) was found and fixed the same way as the GLOBAL case: code inspection, not the harness, since it changed only how long a correct answer took to compute.

All four fixes are reflected in the numbers reported in Section 11. That only the first was actually caught by an automated check is itself a threat to validity worth naming plainly: the verification harness is real and it does its job, but it checks result equivalence, not performance, and it cannot catch an inefficiency that both engines agree is merely slow. A less careful reading of the same benchmark output — one that trusted the harness to have caught everything worth catching, or that trusted an earlier draft's own "architectural limitation" framing at face value — would have reported the pre-fix numbers as a stable finding, twice.

12.6 Negative Results

Two negative results deserve to be stated plainly, because they discipline the paper's overall claim. An earlier version of this section reported a third — TEMPORAL degradation under an open-ended range — which Section 12.2 describes fixing; we do not restate that history a third time here, but note it rather than silently drop the count from three to two.

The first of the two negative results that remain is the memory cost quantified in Section 11.4: SFDB uses 1.3–2.5× more resident memory per fact than the KG baseline at the two larger scales measured. This is the tradeoff that funds every latency advantage reported in this paper, and it is intrinsic to the design rather than an implementation artefact: redundant per-open-set registration is what makes the stalk and temporal indexes cheap to query.

The second is the vacuousness of the sheaf condition in our setting (Section 6.4): the system is, strictly, a presheaf database with a defensive consistency check, not a sheaf database in any technical sense that would constitute a contribution on its own. This does not diminish the practical value of the design: what makes the design work is the context poset, the stalk index, the canonical intermediate representation, and the disciplined restriction vocabulary that comes with them. It does mean that any framing of the paper as a sheaf-theoretic breakthrough would be dishonest, and we do not adopt that framing.

13 Limitations

We describe the design's limitations honestly and in one place so that a reader can weigh what the paper does and does not claim.

13.1 Scale and Deployment

The current prototype is single-node and in-memory. All measurements are at scales of 10 000 facts or below; behaviour at larger scales is extrapolated from the complexity analysis of Section 8 and not directly measured. Global-section computation in the worst case is $O(|\mathcal{C}| \cdot N^2)$, which imposes a natural ceiling on the size of context poset that the system can support on a single node. This ceiling is compounded by a concretely measured one: Section 11.4 reports 1.3–2.5× higher resident memory per fact than the KG baseline, so the single-node memory budget is consumed faster by SFDB than by an equivalent reified store, independent of the gluing worst case. Distributing the store across nodes would require a partitioning strategy for the context poset and a distributed restriction-map protocol; both are outside the scope of this paper.

13.2 Mathematical Assumptions

The formal model assumes a finite context poset equipped with the Alexandrov topology. This is the right setting for the intended workload, but it is also the setting in which the sheaf condition holds vacuously. Nothing in the paper claims a non-trivial sheaf theorem. The gluing operation in the implementation is a defensive consistency check rather than a step in a proof; in the presence of genuinely contradictory local data (which our schema currently forbids), the algorithm would report the conflict rather than resolve it.

13.3 Implementation Limitations

The prototype does not currently support concurrent transactions, on-disk persistence beyond the built-in SQLite backing store, distributed execution, streaming or incremental fact ingestion, or SPARQL compliance. An earlier draft of this paper described the query language as "a small subset of SPARQL"; that overstated it. It is a bespoke query DSL whose surface syntax borrows SPARQL's `SELECT`/`WHERE` keywords and `?var` notation but does not parse SPARQL's

graph-pattern braces or its `FROM` clause, together with a few sheaf-native operations (restriction, stalk lookup). The typed `Query` object used internally by both engines and by the benchmark harness (Section 11) does not depend on this text surface. A real SPARQL front end and a proper transactional layer are engineering work that would be needed for any real deployment but that would not change the paper's structural findings.

13.4 Baseline and Benchmark Limitations

The KG baseline is a research prototype implemented by the same team as the sheaf engine. It is a controlled reference point, not a state-of-the-art RDF store. The baseline is fair for the purpose of isolating the effect of context-indexed storage because both engines share the same serialisation, canonicalisation, and storage backend (SQLite); the only architectural difference is the indexing strategy (triple indexes vs. stalk+context indexes). The evaluation therefore quantifies what disciplined context-indexed storage buys over a strict reification-plus-joins implementation at comparable engineering effort. Repeating the evaluation against a production RDF store on a standard benchmark such as LUBM [6], BSBM, or WatDiv is the natural next step and is called out as future work. The synthetic datasets, while parameterised, may not exhibit all pathologies of organic knowledge graph data.

14 Future Work

Several concrete extensions would improve either the design or the credibility of the evaluation. We list them in the order we consider useful; the first is no longer future work, and we keep the section to record what it was and what came of it.

14.1 A Flat Temporal Index for Unbounded Ranges — Implemented

The GLOBAL-query fix (Section 12.1) was already implemented as of the previous revision of this paper; the same class of gap remained, at that time, for TEMPORAL queries with an open-ended range (Section 12.2): the year-bucketed temporal index had no bounded candidate set to fall back on, so it degraded toward consulting every bucket. This section originally proposed a flat, sorted temporal index (an interval tree, or a sorted array of (start, end) pairs searched by binary search) alongside the bucketed one, so that an unbounded query could binary-search to its starting point and scan forward instead. We have since implemented exactly that (`sfdb.sheaf.indexes.TemporalIndex.facts_ending_after` and its symmetric counterpart `facts_starting_before`); the benchmarked result is a 1.1–1.4× SFDB speedup on unbounded TEMPORAL queries rather than the $\sim 40\times$ slowdown reported previously, described in full in Section 12.2. We record the original proposal here rather than deleting it, since the fact that it turned out to be a small, well-scoped change rather than a genuine architectural limitation is itself part of the paper's evidence about how easy this class of gap is to mistake for a design ceiling.

14.2 Real-World Datasets and Standard Benchmarks

The evaluation uses synthetic data. Repeating it on LUBM [6] and on a subset of DBpedia or Wikidata would provide numbers that can be compared against published results for Jena, GraphDB, and Virtuoso. The LUBM data loader is already implemented (`sfdb.datasets.lubm`); the missing piece is wiring it into the benchmark runner and reporting cross-engine numbers on the LUBM query set.

14.3 Integration With a Production RDF Store

The evaluation in Section 11 compares SFDB only against our own KG baseline; Section 12.5 names this as implementation bias and flags LOOKUP as the query class most likely to overstate SFDB's advantage against a mature engine. As a small, honest check on that risk rather than a full answer to it, we ran a single external-baseline spot-check: the same 1 000-fact stream and the same LOOKUP anchor entity used in Section 11, inserted into `rdflib` [10] (a real, independent, in-memory RDF library, reachable via `scripts/rdflib_spotcheck.py`) and queried with the equivalent SPARQL pattern. SFDB was 12.9× faster than rdflib on this single class at this single scale, both engines returning the same 287 results. We report this as a preliminary, narrowly-scoped data point, not as validation of the paper's broader claims: it is one query class, one scale, and rdflib is itself an in-memory, single-process library rather than a disk-backed, transactionally-serious store like Jena TDB2, Virtuoso, or GraphDB. Extending the comparison to those systems at the scales and query classes reported in Section 11 — rather than this one spot-check — remains the future work needed to report absolute performance against the state of the art.

14.4 Incremental Restriction and Global-Section Maintenance

Restriction maps are computed on demand and cached. Incremental maintenance—updating cached restriction maps and the global-section cache on insert rather than invalidating them—would improve query latency in workloads that mix inserts and queries.

14.5 Concurrent and Distributed Execution

Global-section computation is embarrassingly parallel across independent context sub-trees. A parallel implementation would remove the single-node scalability ceiling. A distributed implementation would additionally require a partitioning strategy for the context poset and an inter-node restriction-map protocol.

14.6 Compression of Contextually Redundant Data

Section 11.4 reports that SFDB uses 1.3–4.8× more resident memory per fact than the KG baseline, because each fact is registered in roughly eight to ten open sets plus four auxiliary indexes. Sections that share a common context or that agree under restriction on a large subtree admit context-aware compression schemes that are not available to a flat triple store: rather than storing a full section reference per open set, a fact could store one canonical reference plus a small bitset of the open sets it belongs to, reconstructed on read. Realising this would directly reduce the memory overhead identified in Section 11.4, which is currently the paper's primary reported cost.

14.7 Hybrid Storage — Revisited

An earlier version of this section proposed a hybrid system that would store facts under both representations and let the optimiser choose the access path per query, on the premise that this would combine SFDB's LOOKUP advantage with a GLOBAL advantage that belonged to the KG baseline. That premise no longer holds: Section 11 reports SFDB faster than the KG baseline on GLOBAL as well as LOOKUP, and Section 12.2 closes the one query shape (unbounded TEMPORAL) where the KG baseline had been competitive. With no latency-motivated reason left to maintain two copies of the data, full dual-representation storage is not the extension we would prioritise: it would double the memory cost identified in Section 11.4 — the paper's one remaining tradeoff — to hedge against a latency gap that the last two fixes in this evaluation have, between them, closed for every class we measure. A narrower version

of the idea could still be worth revisiting if a workload surfaces a genuine KG-favoured access pattern this evaluation does not cover, but we would want that pattern identified first rather than storing everything twice speculatively.

15 Conclusion

We described the Semantic Fact Database, a prototype context-indexed storage system for semantic facts. The system stores each fact atomically as a section over a finite context poset and answers entity-anchored queries through a stalk index that returns complete facts in a single hash lookup. A companion reified triple-store baseline is provided over the same canonical schema, and a mechanised proof of semantic equivalence binds the two engines together: every insert and every query is checked, online, for canonical result equality.

The evaluation, on four query classes and insert throughput at three scales, yields a clear result and a clear tradeoff. SFDB is faster on every measured dimension: inserts by 1.05–1.36×; entity-anchored LOOKUP queries by 23–32× because the stalk index removes the need for triple reassembly; unrestricted GLOBAL queries by 44–95× once that same flat index is used instead of enumerating topology open sets; bounded temporal-interval queries by 1.2–1.4× via the engine's year-bucketed temporal index; and unbounded temporal-interval queries by 1.1–1.2× via a second, flat, binary-searchable temporal index we added specifically because the bucketed index alone cannot narrow an open-ended range. The tradeoff that funds these advantages is memory: SFDB uses 1.3–2.5× more resident memory per fact than the reified-triple baseline at the two larger scales measured. We report this plainly, and it is now the paper's only reported latency-independent tradeoff, following the fix described above. We also report four bugs and performance gaps found and fixed while preparing this evaluation, by three different means: the sheaf engine's LOOKUP path silently used neighbourhood (any-mention) rather than strict-subject match, caught by tightening the cross-engine verification harness itself (Section 7); the GLOBAL open-set enumeration inefficiency that produced an earlier draft's headline negative result was found by inspecting the code path, not by the harness, since both engines already agreed on results before the fix and disagreed only on speed (Section 12.1); a linear-scan identifier decoder in the KG baseline, also found by inspection, which had been inflating the KG engine's own reconstruction cost; and the unbounded temporal-range degradation, found by the same code-inspection route as the GLOBAL fix and closed the same way, by adding an index the design already had room for rather than by redesigning around the limitation (Section 12.2). We treat the fact that our own evaluation needed four rounds of correction, by three different mechanisms, as part of the paper's evidence for the value of that scrutiny rather than as an embarrassment to suppress.

The formal apparatus in the paper is that of presheaves on a finite poset. We do not claim a novel sheaf-theoretic theorem: on the Alexandrov topology that fits our setting, the sheaf condition holds vacuously and could not serve as a technical contribution. The value of the framework is operational. It gives us a disciplined vocabulary for restriction, stalks, and global sections that maps directly onto the components of the implementation, and it gives us a canonical intermediate representation that makes cross-engine equivalence a checkable property rather than an aspiration.

We do not claim that the design supplants production RDF stores. Our KG baseline is a controlled research implementation and the reported speedups are best interpreted as a measurement of what disciplined context-indexed storage buys over a strict reification-plus-joins design at comparable engineering effort. Extending the evaluation to standard benchmarks and to production RDF endpoints is future work. The artifact and reproducibility manifest for every result in the paper are released with the paper.

16 Artifact Availability and Reproduction

The complete source code, datasets, benchmark harness, and reproducibility manifests are released under the MIT License at `https://github.com/BlackSamuron0305/semantic-fact-db`. Table 7 records the exact commit, interpreter, lock-file hash, dataset checksum, and seed behind every number in this paper; `results/paper_suite_reproducibility.json` carries the same manifest alongside the raw benchmark output.

Table 7: Reproducibility metadata for the reported run.

Property	Value
Git commit	`92b99ed`
Python version	`3.14.4`
uv lock hash	`f5166dc09722c41f`
Dataset checksum	`de7d15093ff37d6f`
Random seed	`42`
Generation date	`2026-07-18T11:53:33.247469+00:00`

From a clean clone, four commands reproduce every number and figure in this paper (`sfdb benchmark` with no `-size` argument runs all three scales — 10 timed runs and 2 warm-up iterations per query class per Section 11.1 — and exits non-zero if any cross-engine verification fails):

```
uv sync --group dev
uv run pytest                                  # 358 tests should pass
uv run sfdb benchmark                          # populates results/
uv run python scripts/generate_tables.py       # tables + reproducibility macros
uv run python scripts/generate_figures.py      # figures
cd paper && latexmk -pdf main.tex              # main.pdf
```

Source code, data generators, and benchmark harnesses are released under the MIT License.

A Supplementary Formal Material

This appendix collects formal content referenced from the main text but not required to follow it: definitions that are not cited by number elsewhere in the paper (Appendix A.1), and the standard theorems that follow from the formal model of Section 6 but that this paper does not build on directly (Appendix A.2). The one theorem the paper's argument actually depends on, cross-engine equivalence (Theorem 7.1), is proved in place in Section 7 rather than here.

A.1 Additional Definitions

Definition A.1 (Entity)**.** An *entity* is a distinguished identifier $\varepsilon \in \mathcal{E} \subseteq \mathcal{I}$ with:

- A unique identifier $\mathrm{id}(\varepsilon) \in \mathcal{I}$.
- A semantic type $\mathrm{type}(\varepsilon) \in \textbf{SemanticType}$.
- An optional name $\mathrm{name}(\varepsilon)$.
- Optional attributes $\mathrm{attrs}(\varepsilon) \subseteq \mathcal{V}^*$.

If $f = (i, s, r, \vec{o}, c, k, m)$ is a fact, then $s \in \mathcal{E}$.

Definition A.2 (Relation)**.** A *relation* is a distinguished identifier $\rho \in \mathcal{R} \subseteq \mathcal{I}$ with:

- A unique identifier $\mathrm{id}(\rho) \in \mathcal{I}$.
- An arity $\mathrm{arity}(\rho) \geq 0$.
- Slot types $\mathrm{slots}(\rho) \in \mathbf{SemanticType}^*$.

A fact $f = (i, s, r, \vec{o}, c, k, m)$ is well-typed iff $\mathrm{arity}(r) = |\vec{o}|$ and each ω_j conforms to $\mathrm{slots}(r)_j$.

Definition A.3 (Open Set)**.** An *open set* $U \in \mathcal{T}$ is a subset of X that is declared open. In the Alexandrov topology induced by $(\mathcal{C}, \leq)$, every down-set $\downarrow c = \{d \in \mathcal{C} \mid d \leq c\}$ is open. To each context c we associate $U_c = \{f \in \mathcal{I} \mid \mathrm{context}(f) \leq c\}$. Properties: $U_\top = X$, $U_{c_1} \cap U_{c_2} = U_{c_1 \wedge c_2}$.

Definition A.4 (Knowledge State)**.** A *knowledge state* is $\Sigma = (X, F)$ where X is a finite topological space of fact identifiers and F is a presheaf on X assigning facts to open sets. State transitions: insert $\Sigma \mapsto \Sigma \cup \{f\}$, delete $\Sigma \mapsto \Sigma \setminus \{f\}$, update $\Sigma \mapsto (\Sigma \setminus \{f\}) \cup \{f'\}$.

Definition A.5 (Canonical Query)**.** A *canonical query* is $Q = (\tau, s, r, \vec{o}, c, k_{\min}, n_{\max})$ where: $\tau \in \{\mathrm{FACT}, \mathrm{WALK}, \mathrm{JOIN}, \mathrm{GLUE}\}$ is the query mode, $s \in \mathcal{E} \cup \{*\}$ is the subject (wildcard $*$ matches all), $r \in \mathcal{R} \cup \{*\}$ is the relation, $\vec{o} \in \mathcal{V}^* \cup \{*\}$ is the object pattern, $c \in \mathcal{C} \cup \{*\}$ is the context, $k_{\min} \in [0, 1]$ is the minimum confidence, $n_{\max} \in \mathbb{N}$ is the result limit. The denotation $[\![Q]\!]_\Sigma$ is the set of facts matching Q.

Definition A.6 (Canonical Result)**.** A *canonical result* is $R = [\![Q]\!]_\Sigma$, a set of canonical facts. Two results R_1, R_2 are equivalent iff $|R_1| = |R_2|$ and there exists a bijection $\phi : R_1 \to R_2$ preserving all semantic components up to identifier isomorphism. This bijection-based notion is given here for completeness; it is not the definition the implementation or the equivalence proof use. Section 7 defines *semantic equivalence* (Definition 7.2) as literal equality of order-independent, type-preserving canonical result sets, which is what the benchmark harness checks and what Theorem 7.1 proves; that is the operative definition throughout the rest of the paper.

A.2 Theorems and Proofs

The definitions above and in Section 6.3 satisfy the standard consequences one would expect of a presheaf formalisation. None of the theorems below is cited by number elsewhere in the paper; we record them here as a completeness check on the formal model rather than as load-bearing results, in contrast to Theorem 7.1 (Section 7), which the paper's central claim actually depends on.

A.2.1 Semantic Preservation

Theorem A.1 (Semantic Preservation / Round-trip Correctness)**.** Let $T : \mathcal{F} \to \mathcal{T}(\mathcal{F})$ be the triple decomposition function (reifying an n-ary fact into $2 + n$ triples), and $R : \mathcal{T}(\mathcal{F}) \to \mathcal{F}$ be the reconstruction function. For any semantic fact $f \in \mathcal{F}$:

$$R(T(f)) = f.$$

Proof Sketch. Decomposition T creates triples $(s, \texttt{rdf:type}, \texttt{rdf:Statement})$, $(s, \texttt{rdf:subject}, f.\mathrm{subject})$, $(s, \texttt{rdf:predicate}, f.\mathrm{relation})$, and $(s, \texttt{rdf:object}_j, f.\mathrm{objects}_j)$ for each j. Reconstruction R groups triples by statement ID s and inverts each step. Since T is injective and R is its left inverse on $\mathrm{im}(T)$, the round-trip is identity. □

A.2.2 Functoriality of Restriction

Theorem A.2 (Restriction Functoriality)**.** The restriction maps $\rho_{d,c} : F(d) \to F(c)$ for $c \leq d$ satisfy:

$$\rho_{c,c} = \mathrm{id}_{F(c)}, \qquad \rho_{e,c} = \rho_{d,c} \circ \rho_{e,d} \quad (c \leq d \leq e).$$

Hence $F : \mathcal{C}^{\mathrm{op}} \to \mathbf{Set}$ is a functor (a presheaf).

A.2.3 Sheaf Axioms

As discussed in Section 6.4, these hold vacuously on the Alexandrov topology used throughout this paper; we record their statements for completeness, not as a technical contribution.

Theorem A.3 (Locality)**.** Let F be a presheaf on $(X, \mathcal{T})$, $U \in \mathcal{T}$, $\{U_i\}$ an open cover of U, and $s, t \in F(U)$. If $\rho_{U,U_i}(s) = \rho_{U,U_i}(t)$ for all i, then $s = t$.

Theorem A.4 (Gluing)**.** Let F be a sheaf on $(X, \mathcal{T})$, $U \in \mathcal{T}$, $\{U_i\}$ an open cover of U, and $\{s_i \in F(U_i)\}$ a compatible family: $\rho_{U_i,U_i\cap U_j}(s_i) = \rho_{U_j,U_i\cap U_j}(s_j)$ for all i, j. Then there exists a unique $s \in F(U)$ with $\rho_{U,U_i}(s) = s_i$ for all i.

Proof Sketch. For each $x \in U$, pick i with $x \in U_i$ and define $s(x) = s_i(x)$. Compatibility ensures this is well-defined on overlaps. Uniqueness follows from the locality axiom. □

A.2.4 Canonical Equivalence

Theorem A.5 (Invertibility of the Canonical Mapping)**.** Let $\phi : \mathcal{F} \to \mathfrak{F}$ be the canonical mapping from facts to canonical facts, and $\psi : \mathfrak{F} \to \mathcal{F}$ the inverse mapping. Then:

$$\psi(\phi(f)) = f, \quad \phi(\psi(\mathfrak{f})) = \mathfrak{f}$$

for all $f \in \mathcal{F}$, $\mathfrak{f} \in \mathfrak{F}$.

Cross-engine query equivalence between Σ_{KG} and Σ_{Sheaf} is stated and proved as Theorem 7.1 in Section 7, together with the canonical-form machinery (Definitions 7.1 and 7.2) it depends on; we do not restate it here to avoid a second, weaker copy of the same claim.

A.2.5 Structural Properties

Theorem A.6 (Context Meet as Open Set Intersection)**.** For any $c_1, c_2 \in \mathcal{C}$ in the Alexandrov topology $\mathcal{T}_A$:

$$U_{c_1} \cap U_{c_2} = U_{c_1 \wedge c_2}.$$

Proof. $d \in U_{c_1} \cap U_{c_2}$ iff $d \leq c_1$ and $d \leq c_2$, which is equivalent to $d \leq c_1 \wedge c_2$, i.e., $d \in U_{c_1\wedge c_2}$. □

Theorem A.7 (Preservation of Referential Integrity)**.** For any knowledge state Σ and fact $f \in \Sigma$:

$$\mathrm{subject}(f) \in \mathcal{E}_\Sigma, \quad \mathrm{relation}(f) \in \mathcal{R}_\Sigma, \quad \forall \omega_j \in \vec{o} : \omega_j \in \mathcal{E}_\Sigma \text{ if reference.}$$

Theorem A.8 (Determinism of Query Execution)**.** For a fixed query Q and fixed knowledge state Σ, the result $[\![Q]\!]_\Sigma$ is uniquely determined, independent of execution order, caching state, parallelism, or query history.

Theorem A.9 (Consistency of Global Sections)**.** A family of local sections $\{s_i \in F(U_i)\}$ can be glued to a global section $s \in F(\mathcal{C})$ iff for every pair (i, j):

$$\rho_{U_i,U_i\cap U_j}(s_i) = \rho_{U_j,U_i\cap U_j}(s_j).$$

A.3 Further Detail

Fully worked proofs of every theorem above, pseudocode for the algorithms described in Section 9, and the implementation-to-definition validation table are kept in the repository rather than reproduced here, since they change with the code and would go stale in a typeset PDF:

Proofs `research/proofs/`

Algorithms `research/mathematics/algorithms.md`

Validation table `research/mathematics/validation.md`

References

[1] Renzo Angles. The property graph database model. *Proceedings of the Alberto Mendelzon International Workshop on Foundations of Data Management*, 1912:1–12, 2017.

[2] Renzo Angles and Claudio Gutierrez. Survey of graph database models. *ACM Computing Surveys*, 40(1):1–39, 2008.

[3] Jeremy J. Carroll, Christian Bizer, Pat Hayes, and Patrick Stickler. Named graphs, provenance and trust. In *Proceedings of the 14th International Conference on World Wide Web (WWW)*, pages 613–622. ACM, 2005.

[4] Herbert Edelsbrunner and John Harer. *Computational Topology: An Introduction*. American Mathematical Society, 2010.

[5] Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer, 1999.

[6] Yuanbo Guo, Zhengxiang Pan, and Jeff Heflin. LUBM: A benchmark for OWL knowledge base systems. In *Proceedings of the 4th International Semantic Web Conference (ISWC)*, pages 158–172. Springer, 2005.

[7] Steve Harris and Andy Seaborne. SPARQL 1.1 query language. W3C recommendation, World Wide Web Consortium, 2013.

[8] Frank Manola and Eric Miller. RDF primer. W3C recommendation, World Wide Web Consortium, 2004.

[9] Evan Patterson. Sheaf theory in database theory. In *Proceedings of the 3rd Annual International Symposium on Category Theory and Computer Science*, 2022.

[10] RDFLib Team. rdflib: A python library for working with RDF, 2024.

[11] Michael Robinson. Sheaf theory in data fusion. *SIAM Review*, 62(3), 2020.

[12] David I. Spivak. Functorial data migration. *Information and Computation*, 217:31–51, 2012.